Python Task Manager Subtask Implementation

https://gemini.google.com/app/9a14ca980f938399

User prompt: task_manager.py: ```python from datetime import date def add_task(tasks, title, due_date=None, priority="medium"): """Add a new task to the task list. Args: tasks (list): The list of existing task dictionaries. title (str): The task's title. due_date (date, optional): The task's due date. Defaults to None. priority (str): One of "high", "medium", "low". Defaults to "medium". Returns: list: The updated list of tasks. """ tasks.append({"title": title, "due_date": due_date, "priority": priority, "completed": False}) return tasks def mark_complete(tasks, title): """Mark a task as completed by title. Args: tasks (list): The list of task dictionaries. title (str): The title of the task to mark complete. Returns: bool: True if a matching task was found and updated, False otherwise. """ for task in tasks: if task["title"] == title: task["completed"] = True return True return False def get_overdue_tasks(tasks, current_date): overdue = [] for task in tasks: if task["due_date"] < current_date and not task["completed"]: overdue.append(task) return overdue def calculate_completion_rate(tasks): if not tasks: return 0.0 completed = sum(1 for t in tasks if t["completed"]) return completed / len(tasks) ``` Implement add_subtask(tasks, parent_title, subtask_title). Follow these project conventions explicitly: define TaskError as a subclass of Exception in the same file; if the parent is absent raise TaskError, never return False or None as a failure signal. Match titles ignoring case and leading/trailing spaces. Append a new subtask dictionary to the parent's subtasks list, creating that list if absent. The new dictionary must have exactly title=subtask_title, due_date=None, priority equal to the parent's priority, and completed=False. Return the same updated tasks list object. Keep other tasks unchanged. Give the function a Google-style docstring with Args: and Returns: (and document the exception). Assume valid task dictionaries and a list-valued subtasks field when present; on duplicate matching titles use the first match. Return only TaskError and the function in one Python code block.

Response:
```
class TaskError(Exception):
    """Exception raised for errors in task management operations."""
    pass


def add_subtask(tasks, parent_title, subtask_title):
    """Add a subtask to an existing parent task.

    Args:
        tasks (list): The list of existing task dictionaries.
        parent_title (str): The title of the parent task to add the subtask to.
        subtask_title (str): The title of the new subtask.

    Returns:
        list: The updated list of tasks.

    Raises:
        TaskError: If no parent task matching `parent_title` is found.
    """
    normalized_parent = parent_title.strip().lower()
    for task in tasks:
        if task["title"].strip().lower() == normalized_parent:
            subtask = {
                "title": subtask_title,
                "due_date": None,
```

```
                "priority": task["priority"],
                "completed": False,
            }
            if "subtasks" not in task:
                task["subtasks"] = []
            task["subtasks"].append(subtask)
            return tasks

    raise TaskError(f"Parent task '{parent_title}' not found.")
```
